

Security threats

A security threat is any potential cause of an unwanted event that could harm a network, its data, or the people who rely on it. This dot point names five of them. Your job in the exam is not just to define each one — it's to recognise it inside a case study and explain the damage it does. Let's get the vocabulary straight.

What this key knowledge covers

This summary distils everything you need for **KK08 — Security threats** in Cyber Security. Work through the explanations, then use the worked good-vs-weak answers to see exactly how marks are won and lost. Tick off the revision checklist at the end before a SAC or exam.

Study summary

Threat vs vulnerability vs risk

These three words are *not* synonyms, and a case-study question will often hand you all three in the same paragraph to see whether you can tell them apart. Lock this in now:

The **threat** is a burglar in your neighbourhood. The **vulnerability** is your unlocked back window. The **risk** is how likely the burglar is to try *your* house and how much you'd lose if they did. Fixing the window (the vulnerability) is usually the only thing *you* control — you can't make the burglar disappear.

Denial-of-service (DoS & DDoS) attacks

A **denial-of-service (DoS) attack** deliberately overwhelms a server, service or network with so much traffic or so many bogus requests that it can no longer respond to *legitimate* users. The attacker isn't usually stealing data — they're making the service **unavailable**. When the flood comes from one machine it's a DoS; when it comes from thousands of hijacked devices at once (a **botnet**), it's a **distributed** denial-of-service — **DDoS**, which is far harder to block because there's no single source to switch off.

Imagine 50,000 prank callers ringing a pizza shop's only phone at once. The line is never *broken* — but no real customer can get through to order. That's a DoS. Now imagine the prank callers are spread across thousands of phones in different cities so you can't just block one number. That's a DDoS.

For Clutch, a DDoS at 7 pm on a school night — peak lesson-booking time — means thousands of learners staring at a spinning wheel, instructors unpaid, and parents not getting their "lesson finished" alert. No data is stolen, but the business reputation takes the hit. Use the lab below to *feel* how the attack scales and why mitigation matters.

Mitigation filters the bot traffic at the edge so real learners get through again — exactly what a scrubbing service / firewall rule does in real life.

Rate-limiting, traffic scrubbing services (e.g. Cloudflare-style filtering), firewalls and auto-scaling capacity all reduce DoS impact. You'll meet these properly in KK09 — Reducing risk ; here you only need to *identify and explain* the threat.

Poor credential management

Credentials are the keys to a system: usernames, passwords, API keys, access tokens and certificates. **Credential management** is the whole *process* of creating, storing, sharing, rotating and revoking those keys. When that process is sloppy — that's **poor credential management**, and it's one of the most common ways real organisations get breached.

What "poor" looks like

Clutch's three developers all log in as the same admin account. When something goes wrong, no one knows *who* did it, and you can't disable one person without locking out everyone.

Passwords saved in a shared Google Sheet or hard-coded inside the app's source code. Anyone who sees the file — or the public GitHub repo — owns the system.

A casual instructor leaves, but their login is never disabled. Months later that dormant account is the door an attacker walks through.

The same admin password since 2021, no multi-factor authentication , and it's reused on the founder's personal email too.

Poor credential management is like cutting twenty copies of the building master key, lending them to anyone who asks, never collecting them back when someone quits, and taping a spare under the welcome mat. The locks are fine — it's the *handling of the keys* that's broken.

Malicious software (malware)

Malicious software — malware — is any program written to damage, disrupt or gain unauthorised access to a system. As a *threat* in this dot point, what matters is that it's an **attacker's tool that runs code on a machine it shouldn't**: it can steal Clutch's licence data, encrypt files for ransom, log keystrokes, or quietly turn staff laptops into part of someone else's botnet.

The detailed mechanics of **spyware , viruses, worms and ransomware** are KK07 — Malware. Here in KK08, malware is one threat among five — so be able to *name it as a threat, say how it typically gets in, and describe the harm*. Don't re-write the whole KK07 taxonomy in a KK08 answer; spend your marks on identification and impact.

Outdated software

Outdated software is any operating system, application, library or firmware running an old version that's **missing security patches**. Every time vendors fix a security hole, they publish it (often with a CVE identifier). The moment that fix is public, every unpatched machine becomes a **known, documented, copy-paste target** — attackers literally scan the internet for systems still running the vulnerable version.

It's like a locksmith publicly announcing "every Brand-X lock made before March can be opened with a bent paperclip." Brand-X releases a free replacement lock the same day. If Clutch never

swaps theirs out, every burglar in town now has the instructions — and knows exactly which door to try.

For Clutch this could be an unpatched web server, an old version of the database with a known exploit, or staff laptops still on an end-of-life operating system that no longer receives updates at all. The danger isn't theoretical — outdated software is behind a large share of real-world breaches precisely because the exploit is already written and freely available.

Five threats at a glance

Floods a service with traffic so legitimate users can't get through. DDoS uses many hijacked devices (a botnet) at once.

Sloppy handling of logins & keys: shared accounts, plain-text storage, no rotation, no off-boarding, no MFA.

Attacker's code that runs on a machine to steal, encrypt, spy or hijack. (Full taxonomy = KK07.)

Unpatched OS / apps / firmware with known, published vulnerabilities that attackers actively scan for.

Short, common or reused passwords cracked by brute-force, dictionary or credential-stuffing attacks.

Sort it: threat, vulnerability or consequence?

Drag each statement into the right bin. This is exactly the discrimination examiners test — getting the *category* right is worth marks even before you explain anything.

Command-word decoder

Half of exam technique is doing what the **command word** actually asks. Tap any word to see what it demands and roughly how to spend the marks.

VCE-style questions

Five questions spanning the command words you'll meet. For each: reveal a **weak** response, a **strong** response and the **marking guide**, then self-mark. Your scores feed the dock in the corner.

It's when a hacker attacks a server and it stops working so people can't use the website.

A denial-of-service attack deliberately floods a server or network with so much traffic or so many bogus requests that it can no longer respond to **legitimate** users, making the service **unavailable**. The aim is disruption rather than data theft.

- * 1 Identifies that the service/server is flooded or overwhelmed with traffic/requests.
- * 1 States the effect: legitimate users are denied access / the service becomes unavailable.

A **weak password** is one that is easy to crack — here “clutch123” is short, predictable and dictionary-based, so a brute-force or dictionary attack would break it quickly. **Poor credential management** is about the *practices* for handling credentials: at Clutch the account is *shared* by three developers (so actions can't be traced to a person), and the password has *never been rotated* since 2021. Even a strong password would still be poorly managed if it were shared and never changed — the two problems are separate.

- * 1 Weak password defined (easy to crack: short/common/predictable) + example “clutch123”.

- 1 Poor credential management defined (handling/process: sharing, rotation, storage, revocation).
- 1 Clear distinction drawn + scenario example of the management failure (shared / never rotated).

1. The laptops are old. 2. The email might be bad. These could cause problems for the company.

1. Outdated software. The laptops run an OS that no longer receives security updates, so any newly discovered vulnerability will *never be patched*. Attackers scan for these known, published flaws, making the devices an easy entry point to Clutch's data.

Named exam traps

The threat is the potential bad event; the vulnerability is the weakness it exploits; the consequence is what results. Questions mix all three to test you.

The risk is known, published vulnerabilities that go unfixed — not that the software “looks old” or “runs slowly”. Always mention missing patches.

A DoS is from a single source; a DDoS is distributed across a botnet of many devices, which is why it's far harder to block. Don't use the terms interchangeably.

In KK08, malware is one threat of five. Name it, say how it gets in, and state the harm. The full spyware/virus/worm/ransomware taxonomy belongs to KK07.

Identifying a threat earns the first mark; the rest come from explaining the weakness it exploits and the concrete harm to the people/organisation in the scenario.

KK08 — Security threats, in one place

- DoS / DDoS Flood a service so **real users can't get in**. DDoS = many hijacked devices (botnet) at once → harder to stop. Harm: **unavailability**, lost trust.
- Poor credential mgmt Sloppy **handling** of logins/keys: shared accounts, plain-text storage, no rotation, no off-boarding, no MFA. Not the same as weak passwords.
- Malicious software Attacker's code runs on your machine (steal / encrypt / spy / enslave). Gets in via phishing, downloads, dependencies, USB. Detail = KK07.
- Outdated software Unpatched OS/app/firmware with **known, published vulnerabilities** attackers actively scan for. Fix = patch & update.
- Weak passwords Short / common / reused → cracked by brute-force, dictionary, credential-stuffing. Fix = long, unique, unpredictable + a manager.
- Golden vocab **Threat** = the potential bad event · **Vulnerability** = the weakness it exploits · **Risk** = likelihood × impact.
- Answer shape **P·E·L**: Point (name it) → Explain (how/weakness) → Link (Clutch + impact). Match the command word.

Key terms

Term	Meaning
Phishing email	An instructor opens a fake "lesson invoice.pdf" attachment; it installs a keylogger that captures their Clutch login.

Term	Meaning
Malicious download	A developer grabs a cracked design tool that ships with a trojan, giving an attacker remote access to the dev laptop.
Infected dependency	A compromised open-source library is pulled into Clutch's codebase and ships malware to every device that runs the app.
USB drop	An infected USB stick left in the office car park is plugged in "to see what's on it".

Common traps & misconceptions

Exam trap

△ Exam trap · "name the threat" If a question says "*identify a security threat in this scenario*", it wants the **thing that could go wrong** (e.g. "a denial-of-service attack"), not the weakness ("the server is slow") and not the consequence ("users can't log in"). Name the threat using the study-design vocabulary, then explain the weakness it exploits and the harm it causes.

Exam trap

△ Exam trap · this is NOT the same as "weak passwords" Weak passwords are about the password being **easy to crack** (clutch123). Poor credential management is about the **practices around handling credentials** — sharing, storing, rotating, revoking. A 40-character random password that's emailed around the whole team and never changed is a *strong password with poor credential management*. Examiners love this distinction.

Exam trap

△ Exam trap · "outdated" ≠ "old and slow" The security problem with outdated software is the **missing patches and known vulnerabilities**, not that it "runs slowly" or "looks dated". If you only write "the software is old so it's bad", you've missed the marks. Say: *unpatched* → *known vulnerability* → *attackers exploit a published flaw*.

Exam trap

△ Exam trap · weak vs reused vs poorly-managed Be precise. **Weak** = easy to crack (short/common). **Reused** = strong but used everywhere, so one leak unlocks all. **Poor credential management** = the handling process. A scenario can contain all three at once — say which is which and why each matters.

How to answer exam questions

Read the **command word** first — it sets how much to write. *State/Identify* wants a word or phrase; *Describe* wants features; *Explain* wants reasons and links; *Justify/Evaluate* wants a judgement backed by evidence. Always pull in the **scenario detail** rather than answering in the abstract. The examples below contrast a weak response with a strong one for the same question.

Q1. Describe · 2 marks

Clutch's booking server went offline for two hours during peak evening traffic. No data was accessed. Describe what is meant by a denial-of-service (DoS) attack.

✗ A weak answer

It's when a hacker attacks a server and it stops working so people can't use the website. Vague. "Stops working" could mean anything; no mention of *flooding* or *legitimate users being denied access*. Reads like a guess.

✓ A strong answer

A denial-of-service attack deliberately floods a server or network with so much traffic or so many bogus requests that it can no longer respond to **legitimate** users, making the service **unavailable**. The aim is disruption rather than data theft. Names the mechanism (flooding), the effect (unavailability) and the key idea that *real* users are denied access.

How the marks are awarded

- 1 Identifies that the service/server is flooded or overwhelmed with traffic/requests.
- 1 States the effect: legitimate users are denied access / the service becomes unavailable.

Q2. Explain · 3 marks

At Clutch, the three developers all log in with one shared "admin" account whose password is "clutch123", set in 2021 and never changed. Explain the difference between passwords and poor credential management, using the scenario to illustrate each.

✗ A weak answer

Weak passwords and poor credential management are both about bad passwords. "clutch123" is weak and that's poor management. They should make it stronger. Treats the two as the same thing — the exact error the question is testing. No clear distinction and no separate examples.

✓ A strong answer

A **weak password** is one that is easy to crack — here "clutch123" is short, predictable and dictionary-based, so a brute-force or dictionary attack would break it quickly. **Poor credential management** is about the *practices* for handling credentials: at Clutch the account is *shared* by three developers (so actions can't be traced to a person), and the password has *never been rotated* since 2021. Even a strong password would still be poorly managed if it were shared and never changed — the two problems are separate. Defines each distinctly, gives a separate scenario example for each, and explicitly states they are independent — full marks.

How the marks are awarded

- 1 Weak password defined (easy to crack: short/common/predictable) + example "clutch123".
- 1 Poor credential management defined (handling/process: sharing, rotation, storage, revocation).
- 1 Clear distinction drawn + scenario example of the management failure (shared / never rotated).

Q3. Identify · 4 marks

· Justify Clutch staff use personal laptops still running an operating system that stopped receiving updates last year. One instructor recently opened an email attachment named “invoice.pdf.exe”. Identify two security threats present in this scenario and justify, for each, why it is a threat to Clutch.

✗ A weak answer

1. The laptops are old. 2. The email might be bad. These could cause problems for the company. “Old” isn’t the issue (unpatched is) and “might be bad” isn’t a named threat. No real justification or impact.

✓ A strong answer

1. Outdated software. The laptops run an OS that no longer receives security updates, so any newly discovered vulnerability will *never be patched*. Attackers scan for these known, published flaws, making the devices an easy entry point to Clutch’s data. **2. Malicious software (malware).** The attachment “invoice.pdf.exe” is an executable disguised as a document — a classic phishing payload. If run, it could install a keylogger or trojan, capturing the instructor’s Clutch login or giving an attacker remote access to learner records. Two correctly named threats, each justified by the weakness it exploits AND the concrete harm to Clutch.

How the marks are awarded

- 1 Outdated software identified (unpatched OS / no updates).
- 1 Justified: known vulnerabilities remain unfixed exploited for access.
- 1 Malicious software identified (disguised executable / phishing payload).
- 1 Justified: could install malware steal credentials / data / remote access.

Q4. Compare · 3 marks

Compare a DoS attack with a DDoS attack.

✗ A weak answer

A DoS is a denial of service and a DDoS is a distributed one. They are both attacks on servers. States the names but not the meaningful difference (one source vs many) or why DDoS is harder to stop. No genuine comparison.

✓ A strong answer

Similarity: both aim to make a service unavailable to legitimate users by overwhelming it with traffic or requests. **Difference:** a *DoS* attack comes from a **single** source, so it can often be blocked by filtering that one address; a *DDoS* is **distributed** across thousands of hijacked devices (a botnet), so there is no single source to block, making it far harder to defend against and capable of generating much greater traffic. Gives a shared purpose AND a clear point-of-difference (single vs many sources) with the consequence (harder to stop).

How the marks are awarded

- 1 Similarity: both flood a service to deny access to legitimate users.
- 1 Difference: DoS = single source; DDoS = many distributed sources / botnet.
- 1 Consequence of the difference: DDoS is harder to block / larger scale.